Softwareparadigmen SS 2025, Übungsblatt 2
Gruppe 90

Khaled Elnaggar 12033937,
Ilma Bajramovic,
Nhat Minh Hoang 12029170

Beispiel 1 1/1

1)Below is the implementation of the function `implies(a,b)`.

```
δimplies(a,b) =
  if isFalse?(a) then
    T
  else
    b
```

The interpretation of `implies(a,b)` under the given environment ω and δ proceeds as follows.

$$
\begin{aligned}
&I(\delta, \omega, \underline{\texttt{implies(a,b)}}) \\
&= I(\delta, \omega^1, \underline{\texttt{if isFalse?(a) then }} li_2 \underline{\texttt{ else }} li_4) \\
&\quad \text{with } \omega^1 : \\
&\quad \omega 1(\underline{\texttt{a}}) = I(\delta, \omega, \underline{\texttt{a}}) = \omega(\underline{\texttt{a}}) = T \\
&\quad \omega 1(\underline{\texttt{b}}) = I(\delta, \omega, \underline{\texttt{b}}) = \omega(\underline{\texttt{b}}) = T \\
&= I(\delta, \omega 1, \underline{\texttt{b}}) \\
&\quad \text{since } I(\delta, \omega^1, \underline{\texttt{isFalse(a)}}) = \mathrm{F} \\
&= \omega 1(\underline{\texttt{b}}) \\
&= T
\end{aligned}
$$

2) Below is the implementation of the function `any(l)`.

```
δany(l) =
  if empty?(l) then
    F
  else
    or(first(l), any(rest(l)))
```

The interpretation of `any(l)` under the given environment ω and δ proceeds as follows:

$$
\begin{aligned}
& I(\delta, \omega, \underline{\texttt{any}}(l)) \\
& = I(\delta, \omega^1, \underline{\texttt{if empty?(l) then}}\ la_2\ \underline{\texttt{else}}\ la_4) \\
& \quad \textit{with } \omega^1: \\
& \quad \omega 1(\underline{\texttt{l}}) = I(\delta, \omega, \underline{\texttt{l}}) = \omega(\underline{\texttt{l}}) = [\mathrm{F}, \mathrm{T}] \\
& = I(\delta, \omega 1, \underline{\texttt{or(first(l),any(rest(l)))}}) \\
& \quad \textit{since } I(\delta, \omega 1, \underline{\texttt{empty(l)}}) = \mathrm{F} \\
& = or(I(\delta, \omega 1, \underline{\texttt{first(l)}}), I(\delta, \omega 1, \underline{\texttt{any(rest(l))}})) \\
& = or(\textit{first}(I(\delta, \omega 1, \underline{\texttt{l}})), I(\delta, \omega 1, \underline{\texttt{any(rest(l))}})) \\
& = or(\textit{first}(\omega 1(l)), I(\delta, \omega 1, \underline{\texttt{any(rest(l))}})) \\
& = or(\textit{first}([\mathrm{F}, \mathrm{T}]), I(\delta, \omega 1, \underline{\texttt{any(rest(l))}})) \\
& = or(\mathrm{F}, I(\delta, \omega 1, \underline{\texttt{any(rest(l))}})) \quad \text{(see NR 1)} \\
& = or(\mathrm{F}, \mathrm{T}) \\
& = \mathrm{T}
\end{aligned}
$$

$$
\begin{aligned}
& \textbf{NR 1:}\ I(\delta, \omega 1, \underline{\texttt{any(rest(l))}}) \\
& = I(\delta, \omega^2, \underline{\texttt{if empty?(l) then}}\ la_2\ \underline{\texttt{else}}\ la_4) \\
& \quad \textit{with } \omega^2: \\
& \quad \omega 2(\underline{\texttt{l}}) = I(\delta, \omega 1, \underline{\texttt{rest(l)}}) = \textit{rest}(I(\delta, \omega 1, \underline{\texttt{l}})) = \textit{rest}([\mathrm{F}, \mathrm{T}]) = [\mathrm{T}] \\
& = I(\delta, \omega 2, \underline{\texttt{or(first(l),any(rest(l)))}}) \\
& \quad \text{since } I(\delta, \omega 2, \underline{\texttt{empty(l)}}) = \mathrm{F} \\
& = or(I(\delta, \omega 2, \underline{\texttt{first(l)}}), I(\delta, \omega 2, \underline{\texttt{any(rest(l))}})) \\
& = or(\textit{first}(I(\delta, \omega 2, \underline{\texttt{l}})), I(\delta, \omega 2, \underline{\texttt{any(rest(l))}})) \\
& = or(\textit{first}(\omega 2(\underline{\texttt{l}})), I(\delta, \omega 2, \underline{\texttt{any(rest(l))}})) \\
& = or(\textit{first}([\mathrm{T}]), I(\delta, \omega 2, \underline{\texttt{any(rest(l))}})) \\
& = or(\mathrm{T}, I(\delta, \omega 2, \underline{\texttt{any(rest(l))}})) \quad \text{(see NR 2)} \\
& = or(\mathrm{T}, \mathrm{F}) \\
& = \mathrm{T}
\end{aligned}
$$

$$
\begin{aligned}
& \textbf{NR 2:}\ I(\delta, \omega 2, \underline{\texttt{any(rest(l))}}) \\
& = I(\delta, \omega^3, \underline{\texttt{if empty?(l) then}}\ la_2 \underline{\texttt{else}} la_4) \\
& \quad \textit{with } \omega^3: \\
& \quad \omega 3(\underline{\texttt{l}}) = I(\delta, \omega 2, \underline{\texttt{rest(l)}}) = \textit{rest}(I(\delta, \omega 2, \underline{\texttt{l}})) = \textit{rest}([\mathrm{T}]) = [] \\
& = I(\delta, \omega 3, \mathrm{F}) \\
& = \mathrm{F} \\
& \quad \textit{since } I(\delta, \omega 3, \underline{\texttt{empty(l)}}) = \mathrm{T}
\end{aligned}
$$

Beispiel 2 1.5/2

Induction proof of the `pow(a,b)` function:

symbolic interpretation

1. Base case: $\omega(y) = 0$

$$
\begin{aligned}
& I(\delta, \omega, \underline{\texttt{pow}(x, y)}) \\
= & I(\delta, \omega, \underline{\texttt{if eq?(y,0) then }} 1 \underline{\texttt{ else }} mul(x, pow(x, minus(y, 1))) \\
= & I(\delta, \omega, 1) \\
& \quad since\ I(\delta, \omega, \underline{\texttt{eq?}(y, 0)}) \implies eq?(\omega(y), 0) \implies eq?(0, 0) \to \text{true} \\
= & 1 \\
= & \omega(x)^0 \\
= & \omega(x)^{\omega(y)}
\end{aligned}
$$

2. Hypothesis to prove:

$$\forall \omega \in \text{ENV} : I(\delta, \omega, \text{pow}(x, y)) = \omega(x)^{\omega(y)} \quad \text{if} \quad \omega(x) \geq 0, \omega(y) \geq 0$$

3. Inductive step: $\omega(y) = k + 1$

$$
\begin{aligned}
& I(\delta, \omega, \underline{\texttt{pow(x,y)}} \\
= & I(\delta, \omega, \underline{\texttt{if eq?(y,0 then }} 1 \underline{\texttt{ else }} \texttt{mul(x,pow(x,minus(y,1)))} \\
= & I(\delta, \omega, \underline{\texttt{mul(x, pow(x, minus(y, 1)))}}) \\
& \quad since\ I(\delta, \omega, \underline{\texttt{eq?(y, 0)}}) \implies eq?(\omega(y), 0) \implies eq?(k+1, 0) \to false \\
= & I(\delta, \omega', \underline{\texttt{mul(a, pow(a, b))}} \\
& \quad \omega'(a) = I(\delta, \omega, \underline{\texttt{x}}) \implies \omega(x) \\
& \quad \omega'(b) = I(\delta, \omega, \underline{minus(\omega(y), \omega(1))} \implies \omega(y) - \omega(1) = k + 1 - 1 = k \\
= & I(\delta, \omega'', \underline{\texttt{mul(a, b)}}) \\
& \quad \omega''(a) = I(\delta, \omega', \underline{\texttt{a}}) \implies \omega'(a) \implies \omega(x) \\
& \quad \omega''(b) = I(\delta, \omega', \underline{\texttt{pow(a,b)}}) \implies pow(\omega'(a), \omega'(b)) = \omega'(a)^{\omega'(b)} = \omega(x)^k = \omega(x)^{\omega(y)} = \quad \omega(x) * \omega(x)^k \\
= & \omega(x)^{k+1} \\
= & \omega(x)^{\omega(y)}
\end{aligned}
$$

Induction proof of the `div(a,b)` function:

1. Base case: $\omega(x) = 0$

$$
\begin{aligned}
& I(\delta, \omega, \underline{\texttt{div}(x, y)}) \\
= & I(\delta, \omega, \underline{\texttt{if lt?(x,y) then }} 0 \underline{\texttt{ else }} \texttt{plus(div(minus(x,y),y),1)} \\
= & I(\delta, \omega, 0) \\
& \quad since\ I(\delta, \omega, \underline{\texttt{lt?}(x, y)}) \implies lt?(\omega(x), \omega(y)) \implies lt?(0, \omega(y)) \to true \\
= & 0 < \omega(y) \to true,\ since\ \omega(y) > 0
\end{aligned}
$$

2. Hypothesis to prove:

$$\forall \omega \in \text{ENV} : \ I(\delta, \omega, \text{div}(\mathbf{x}, \mathbf{y})) = \lfloor \omega(\mathbf{x}) \div \omega(\mathbf{y}) \rfloor \quad \text{if } \omega(\mathbf{x}) \geq 0,\ \omega(\mathbf{y}) > 0$$

3. Inductive step $\omega(x) = k + 1$

$$
\begin{aligned}
& I(\delta, \omega, \mathrm{div}(x, y)) \\
= & I(\delta, \omega, \underline{\texttt{if lt?(x,y) then 0 else plus(div(minus(x,y),y),1}} \\
= & I(\delta, \omega, \underline{\texttt{plus(div(minus(x,y),y),1}} \\
& \quad \textit{Case 1: } \omega(x) < \omega(y)\ I(\delta, \omega, \underline{\texttt{lt?(x,y)}}) \implies lt?(\omega(x), \omega(y)) \implies k + 1 < \omega(y) \implies 0 \to true \\
& \quad \textit{Case 2: } \omega(x) \geq \omega(y)\ I(\delta, \omega, \underline{\texttt{lt?(x,y)}}) \implies lt?(\omega(x), \omega(y)) \implies k + 1 > \omega(y) \to false
\end{aligned}
$$

base case

$$
\begin{aligned}
= & I(\delta, \omega, \underline{\texttt{plus(div(minus(x,y),y),1)}}) \\
= & I(\delta, \omega', \underline{\texttt{plus(div(a,b),1)}} \\
& \quad \omega'(a) = I(\delta, \omega, \underline{\texttt{minus(x,y)}} \implies minus(\omega(x), \omega(y)) minus(k + 1, \omega(y)) \implies k + 1 - \omega(y) \\
& \quad \omega'(b) = I(\delta, \omega, y) \implies \omega(y) \\
= & I(\delta, \omega, \underline{\texttt{plus(a, 1)}}) \\
& \quad \omega''(a) = I(\delta, \omega', \underline{\texttt{div(a,b)}}) \implies div(\omega'(a), \omega'(b)) \implies \omega'(a) \div \omega'(b) \implies (k + 1 - \omega(y)) \div \omega'(y)) \\
= & \omega''(a) + 1 \\
= & ((k + 1) \div \omega(y)) + 1 \\
= & \frac{k}{\omega(y)} + \frac{1}{\omega(y)} - \frac{\omega(y)}{\omega(y)} + 1 \\
= & \frac{k}{\omega(y)} + \frac{1}{\omega(y)} - 1 + 1 \\
= & \frac{k + 1}{\omega(y)} \\
= & \frac{\omega(x)}{\omega(y)} = \omega(x) \div \omega(y)
\end{aligned}
$$

Beispiel 3

1. (sum(as) != 0) == !allZero(as)

Lemma 1: `(x == 0 && y == 0) == (x + y == 0)`
Lemma 2: `(x != 0 || y != 0) == (x + y != 0)` prove lemma
Base: `as == Nil`

```
sum(Nil) != 0
sum(Nil) != sum(Nil)          def. sum. r.l.
false                         logic/arithm
!true                         logic/arithm
!allZero(Nil)                 def allZero. r.l.
```

Induction step: Show the assumption holds for `a::as`

```
sum(a::as) != 0
a + sum(as) != 0              def sum. l.r.
a != 0 || sum(as) != 0        Lemma 2 r.l.
a != 0 || !allZero(as)        Hypoth. l.r.
!(a == 0 || allZero(as))      De Morgan's law
!allZero(a::as)               def. allZero. r.l
```

applied de morgan wrong!

0.5/1

2. bigOr(push(as, bs)) == bigOr(as) || bigOr(bs)

Base: `as == Nil`

```
bigOr(push(Nil, bs))
bigOr(bs)                    def. push. l.r
false   || bogOr(bs)         logic/arithm
bogOr(Nil) || bigOr(bs)      def bigOr r.l
```

Induction step: Show the assumption holds for `a::as`

```
bigOr(push(a::as, bs))
bigOr(a::push(as, bs))          def. push. l.r.
a || bigOr(push(as, bs))        def. bigOr. l.r.
a || bigOr(as) || bigOr(bs)     Hypoth. l.r.
bigOr(a::as) || bigOr(bs)       def. bigOr. r.l.
```

1/1

3. bigOr(as) == bigOr(evens(as)) || bigOr(odds(bs))

Base: `as == Nil`

```
bigOr(Nil)
false                                       def. bigOr. l.r
false   || false                            logic/arithm
bogOr(Nil) || false                         def bigOr r.l
bogOr(Nil) || bigOr(Nil)                    def. bigOr. r. l.
bogOr(evens(Nil)) || bigOr(Nil)             def. evens. r. l.
bogOr(evens(Nil)) || bigOr(odds(Nil))       def. odds. r. l.
```

missing second base case

Induction step: Show the assumption holds for `a::y::as`

```
bigOr(a::y::as)
a || bigOr(y::as)                                   def bigOr. l.r.
a || y || bigOr(as)                                 def bigOr. l.r.
a || bigOr(evens(as)) || y || bigOr(odds(as))       Hypoth. l.r.
bigOr(a::evens(as)) || y || bigOr(odds(as))         ~~def. evens. r.l~~.
bigOr(a::evens(as)) || bigOr(y::odds(as))           ~~def. odds. r.l~~.
bigOr(evens(a::y::as)) || bigOr(y::odds(as))        def. evens r.l.
bigOr(evens(a::y::as)) || bigOr(odds(a::y::as))     def. odds r.l.
```

bigOr!

0.75/1